

# md-pdf

*A comprehensive guide*

tschinz - 2026-01-22 - 0.1.0

[documentation](#) [markdown](#) [pdf-conversion](#) [typst](#) [user-guide](#) [configuration](#) [templates](#)

## Contents

|       |                                                              |    |
|-------|--------------------------------------------------------------|----|
| 1     | Overview .....                                               | 3  |
| 1.1   | What is md-pdf? .....                                        | 3  |
| 1.2   | Key Benefits .....                                           | 4  |
| 2     | Installation .....                                           | 4  |
| 2.1   | Prerequisites .....                                          | 4  |
| 2.1.1 | 1. Rust Toolchain (Optional, for building from source) ..... | 4  |
| 2.1.2 | 2. Typst CLI (Required) .....                                | 4  |
| 2.1.3 | 3. Fonts .....                                               | 4  |
| 2.2   | Installing md-pdf .....                                      | 4  |
| 2.2.1 | First Run & Auto-Setup .....                                 | 5  |
| 2.2.2 | Verification .....                                           | 5  |
| 3     | Configuration System .....                                   | 5  |
| 3.1   | Auto-Configuration .....                                     | 5  |
| 3.2   | Creating Configuration Files .....                           | 5  |
| 3.2.1 | Automatic Creation .....                                     | 5  |
| 3.2.2 | Manual Configuration .....                                   | 6  |
| 3.3   | Configuration File Locations .....                           | 6  |
| 3.4   | Configuration File Content .....                             | 6  |
| 3.5   | Configuration Options Reference .....                        | 7  |
| 3.6   | Viewing Configuration .....                                  | 7  |
| 4     | Templates System .....                                       | 7  |
| 4.1   | Built-in Templates .....                                     | 7  |
| 4.2   | Template Selection Priority .....                            | 7  |
| 4.3   | Template Variables & Data Passing .....                      | 8  |
| 4.3.1 | System Variables .....                                       | 8  |
| 4.3.2 | Front Matter Variables .....                                 | 8  |
| 4.3.3 | Custom Fields .....                                          | 8  |
| 4.4   | Using Variables in Templates .....                           | 9  |
| 4.4.1 | Basic Variable Access .....                                  | 9  |
| 4.4.2 | Advanced Variable Processing .....                           | 9  |
| 4.4.3 | Variable Validation and Defaults .....                       | 9  |
| 4.5   | Custom Template Development .....                            | 9  |
| 4.5.1 | Creating Custom Templates .....                              | 9  |
| 5     | Front Matter Support .....                                   | 10 |

|                                                    |    |
|----------------------------------------------------|----|
| 5.1 What is Front Matter? .....                    | 10 |
| 5.2 Basic Front Matter .....                       | 10 |
| 5.3 Supported Data Types .....                     | 10 |
| 5.3.1 Strings .....                                | 10 |
| 5.3.2 Numbers .....                                | 10 |
| 5.3.3 Booleans .....                               | 11 |
| 5.3.4 Arrays/Lists .....                           | 11 |
| 5.3.5 Objects/Dictionaries .....                   | 11 |
| 5.3.6 Dates and Times .....                        | 11 |
| 5.4 Field Precedence and Smart Defaults .....      | 12 |
| 5.5 Front Matter Example .....                     | 12 |
| 6 Usage Guide .....                                | 12 |
| 6.1 Basic Commands .....                           | 12 |
| 6.1.1 Essential Operations .....                   | 13 |
| 6.1.2 Information and Configuration Commands ..... | 13 |
| 6.1.3 Link Checking and Validation .....           | 13 |
| 6.1.3.1 Sample Output .....                        | 13 |

## 1 Overview



### 1.1 What is md-pdf?

**md-pdf** is a powerful, lightweight command-line tool that converts Markdown files into professional PDF documents. Built with Rust and powered by the Typst typesetting system, it provides fast, high-quality document generation.

md-pdf bridges the gap between Markdown's simplicity and PDF's professional presentation. It's designed for developers, writers, researchers, and anyone who needs to create beautiful documents from Markdown source files.

## 1.2 Key Benefits

- 🚀 **Lightning Fast:** PDF generated with Typst, minimal installation footprint
- 🎨 **Professional Output:** High-quality typography and layout
- ⚙️ **Zero Configuration:** Works out of the box with intelligent defaults
- 🛠️ **Highly Customizable:** Flexible template system and configuration options
- 👁️ **Live Preview:** Real-time PDF updates with watch mode
- 🔗 **Link Validation:** Built-in checking for external links to ensure document quality
- 📄 **Rich Metadata:** Comprehensive front matter support

## 2 Installation

### 2.1 Prerequisites

Before installing md-pdf, you need:

#### 2.1.1 1. Rust Toolchain (Optional, for building from source)

If you plan to build from source:

```
# Install rustup if needed
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Restart shell or source the environment
source $HOME/.cargo/env

# Verify installation
rustc --version
cargo --version
```

Visit [rust-lang.org](https://rust-lang.org) for more details.

#### 2.1.2 2. Typst CLI (Required)

Typst is the typesetting engine that powers PDF generation:

```
# Install via cargo (recommended)
cargo install typst-cli

# Verify installation
typst --version
```

Visit [typst.app](https://typst.app) for more details.

#### 2.1.3 3. Fonts

The default template **simple** uses some fonts which are optional.

- [Iosevka](#)
- [Fira Sans](#)
- [JetBrains Mono](#)

## 2.2 Installing md-pdf

Install directly from the Git repository using cargo:

md-pdf

```
# Install latest stable release
cargo install --git https://github.com/tschinz/md-pdf

# Install specific version/tag
cargo install --git https://github.com/tschinz/md-pdf --tag v0.0.1
```

### 2.2.1 First Run & Auto-Setup

After installation, md-pdf is ready to use immediately:

```
# Convert your first document (triggers auto-setup)
md-pdf README.md

# Check what was automatically created
md-pdf --show-config

# List available templates
md-pdf --list-templates
```

On first run, md-pdf automatically creates:

- **Configuration file:** `~/.config/md-pdf/config.toml`
- **Templates directory:** `~/.config/md-pdf/templates/`
- **Template files:** `none.typ` and `simple.typ`

### 2.2.2 Verification

Verify your installation works correctly:

```
# Check version
md-pdf --version

# Test conversion with a simple document
echo "# Test Document" > test.md
echo "This is a test." >> test.md
md-pdf test.md

# Check if PDF was created
ls -la test.pdf
```

## 3 Configuration System

md-pdf uses a sophisticated configuration system that provides intelligent defaults while allowing complete customization.

### 3.1 Auto-Configuration

The tool automatically configures itself on first use:

- Creates a configuration file with sensible defaults
- Sets up a templates directory with proper paths
- Installs basic template files with full content
- No manual setup required!

### 3.2 Creating Configuration Files

#### 3.2.1 Automatic Creation

Configuration is created automatically on first use, but you can manually create it:

md-pdf

```
# Create default configuration file
md-pdf --create-config
```

This creates `~/.config/md-pdf/config.ron` with:

- Proper templates directory path
- Sensible defaults for all options
- Complete template files (none.typ and simple.typ)
- Detailed comments explaining each option

### 3.2.2 Manual Configuration

You can also create configurations in specific locations:

```
# Project-specific configuration
echo '(default_template: Some("custom"))' > ./md-pdf.ron

# User-specific override
mkdir -p ~/.config/md-pdf
md-pdf --create-config
```

## 3.3 Configuration File Locations

md-pdf searches for configuration files in this priority order:

1. `./md-pdf.ron` - Project-specific configuration (highest priority)
2. `<binary-dir>/md-pdf.ron` - Portable installation configuration
3. `~/md-pdf.ron` - User home directory
4. `~/.config/md-pdf.ron` - XDG standard location
5. `~/.config/md-pdf/config.ron` - **Recommended default location**
6. `~/.config/zas/md-pdf.ron` - Workflow integration (lowest priority)

## 3.4 Configuration File Content

The configuration file uses RON (Rust Object Notation) format, which is human-readable and easy to edit:

```
(
  // Path to templates directory (absolute path recommended)
  // This directory contains .typ template files
  templates_dir: Some("/Users/username/.config/md-pdf/templates"),

  // Default template to use when none specified
  // Available options: "none", "simple", or custom template name
  default_template: Some("simple"),

  // Default language for documents
  // Affects typography, hyphenation, and formatting
  default_language: Some("en"),

  // Generate table of contents by default
  // Can be overridden per document via front matter
  default_toc: Some(true),

  // Default author name for documents without explicit author
  // Used when no author specified in front matter
  default_author: Some("Your Name Here"),
)
```

### 3.5 Configuration Options Reference

| Field                   | Type                         | Description                         | Default Value                     | Examples                        |
|-------------------------|------------------------------|-------------------------------------|-----------------------------------|---------------------------------|
| <b>templates_dir</b>    | <b>Option&lt;PathBuf&gt;</b> | Directory containing template files | <b>~/.config/md-pdf/templates</b> | Custom template collections     |
| <b>default_template</b> | <b>Option&lt;String&gt;</b>  | Template name to use by default     | <b>"simple"</b>                   | <b>"academic", "corporate"</b>  |
| <b>default_language</b> | <b>Option&lt;String&gt;</b>  | Document language code              | <b>"en"</b>                       | <b>"de", "fr", "es"</b>         |
| <b>default_toc</b>      | <b>Option&lt;bool&gt;</b>    | Include table of contents           | <b>true</b>                       | Per-document override available |
| <b>default_author</b>   | <b>Option&lt;String&gt;</b>  | Default document author             | <b>"User"</b>                     | Your name or organization       |

### 3.6 Viewing Configuration

Check your current configuration:

```
# Show all configuration details
md-pdf --show-config

# Output example:
# Configuration Information
# =====
# Templates directory: /Users/username/.config/md-pdf/templates
# Default template: simple
# Default language: en
# Default TOC: true
# Default author: Your Name
```

## 4 Templates System

The template system controls the visual appearance and layout of your PDF documents. md-pdf includes built-in templates and supports unlimited custom templates.

### 4.1 Built-in Templates

- **none** Template
- **simple** Template

### 4.2 Template Selection Priority

Templates are chosen in this hierarchical order:

1. **Command-line argument: -t template\_name** (highest priority)
2. **Front matter: template: "template\_name"**
3. **Configuration default: default\_template** setting
4. **System fallback: "none"** template (lowest priority)

Example:

```
# Command-line overrides everything
md-pdf doc.md -t simple # Uses 'simple' regardless of config/frontmatter
```

4.3 Template Variables & Data Passing

Templates receive comprehensive data from the conversion process. Understanding these variables is crucial for creating custom templates.

4.3.1 System Variables

These variables are automatically provided by md-pdf:

| Variable        | Type   | Description                     | Example Value                               |
|-----------------|--------|---------------------------------|---------------------------------------------|
| filepath        | String | Source markdown file path       | <code>"/path/to/document.md"</code>         |
| default_author  | String | Author from configuration       | <code>"Your Name"</code>                    |
| language        | String | Document language               | <code>"en", "de", "fr"</code>               |
| toc             | String | Table of contents flag          | <code>"true"</code> or <code>"false"</code> |
| has_frontmatter | String | Front matter presence indicator | <code>"true"</code> or <code>"false"</code> |

4.3.2 Front Matter Variables

All YAML front matter fields become template variables. Common fields:

| Field   | Variable Name | Type   | Example Value                  | Usage            |
|---------|---------------|--------|--------------------------------|------------------|
| title   | title         | String | <code>"My Document"</code>     | Document title   |
| author  | author        | String | <code>"John Doe"</code>        | Document author  |
| date    | date          | String | <code>"2024-01-22"</code>      | Publication date |
| version | version       | String | <code>"1.0.0"</code>           | Document version |
| tags    | tags          | Array  | <code>["docs", "guide"]</code> | Document tags    |

4.3.3 Custom Fields

Any YAML field in front matter becomes a template variable:

```
---
# Custom fields become template variables
client: "ACME Corporation"
contract_number: "CON-2024-001"
review_date: "2024-02-01"
budget: 50000
approved: true
reviewers:
  - "Alice Johnson"
  - "Bob Smith"
---
```

These become available as:

- `sys.inputs.client` → `"ACME Corporation"`
- `sys.inputs.contract_number` → `"CON-2024-001"`
- `sys.inputs.budget` → `50000`
- `sys.inputs.approved` → `"true"`
- `sys.inputs.reviewers` → `["Alice Johnson", "Bob Smith"]`

## 4.4 Using Variables in Templates

Templates use Typst syntax to access and manipulate variables:

### 4.4.1 Basic Variable Access

```
// Access system variables
#if sys.inputs.toc == "true" [
  #outline(title: "Table of Contents", depth: 3)
  #pagebreak()
]

// Use author with configuration fallback
#let doc_author = sys.inputs.at("author", default: sys.inputs.default_author)
```

### 4.4.2 Advanced Variable Processing

```
// Process arrays (tags, keywords, etc.)
#if "tags" in sys.inputs [
  *Tags:* #sys.inputs.tags.join(", ")
]
```

### 4.4.3 Variable Validation and Defaults

```
// Validate required fields
#let required_fields = ("title", "author", "version")
#for field in required_fields [
  #if field not in sys.inputs [
    #text(fill: red)[Error: Required field '#field' missing from front matter]
  ]
]

// Provide smart defaults
#let doc_title = sys.inputs.at("title", default: "Untitled Document")
#let doc_author = sys.inputs.at("author", default: sys.inputs.default_author)
#let doc_version = sys.inputs.at("version", default: "1.0")
```

## 4.5 Custom Template Development

### 4.5.1 Creating Custom Templates

1. Navigate to templates directory:

```
cd ~/.config/md-pdf/templates
```

2. Create new template file:

```
touch my-custom-template.typ
```

3. Add template description (recommended):

```
// My Custom Academic Template
// Professional template for academic papers and research documents
// Supports: citations, figures, academic formatting, multiple authors
// Created: 2024-01-22
// Last modified: 2024-01-22

// Your template code here...
```

4. Use your template:

```
md-pdf document.md -t my-custom-template
```

## 5 Front Matter Support

Front matter provides document metadata and per-document configuration through YAML headers at the beginning of Markdown files.

### 5.1 What is Front Matter?

Front matter is a YAML block at the very beginning of a Markdown file, enclosed by triple dashes (---). It contains metadata and configuration options that control how the document is processed and formatted.

### 5.2 Basic Front Matter

The simplest front matter includes core document information:

```
---
title: "My Document"
author: "John Doe"
date: "2024-01-22"
tags:
  - tag1
  - tag2
---
# Document content starts here...
```

### 5.3 Supported Data Types

md-pdf handles all standard YAML data types with intelligent processing:

#### 5.3.1 Strings

```
# Simple strings
title: "Document Title"
description: "Single quoted string"

# Multi-line strings (preserve line breaks)
abstract: |
  This is a multi-line string
  that preserves line breaks.
  Perfect for abstracts, descriptions,
  or detailed explanations.

# Folded strings (join lines with spaces)
summary: >
  This is a folded string that joins
  multiple lines with spaces, creating
  a single paragraph of text.

# Escaped strings
special_chars: "Quotes: \"Hello\", Backslash: \\"
```

#### 5.3.2 Numbers

```
# Integers
version_number: 2
revision: 3
page_count: 42
budget: 50000
```

md-pdf

```
# Floating point
price: 29.99
rating: 4.5
percentage: 95.7
```

### 5.3.3 Booleans

```
toc: true
draft: false
confidential: true
approved: false
published: true
```

### 5.3.4 Arrays/Lists

```
# Simple arrays
tags: ["markdown", "pdf", "documentation"]
languages: ["en", "es", "fr"]
```

```
# Multi-line arrays
keywords:
  - technical writing
  - automation tools
  - document processing
  - workflow optimization
```

```
# Mixed type arrays
scores: [95, 87.5, "excellent", true]
```

```
# Nested arrays
matrix:
  - [1, 2, 3]
  - [4, 5, 6]
  - [7, 8, 9]
```

### 5.3.5 Objects/Dictionaries

```
# Simple objects
contact:
  name: "John Doe"
  email: "john@example.com"
  phone: "+1-555-0123"
```

```
# Nested objects
project_info:
  name: "Documentation Project"
  timeline:
    start: "2024-01-01"
    end: "2024-06-30"
  budget:
    allocated: 100000
    spent: 75000
  team:
    lead: "Jane Smith"
    members: ["Alice", "Bob", "Charlie"]
```

### 5.3.6 Dates and Times

```
# ISO 8601 dates
date: "2024-01-22"
created: 2024-01-22T10:30:00Z
```

md-pdf

```
deadline: "2024-12-31T23:59:59Z"
```

```
# Natural language dates (processed as strings)
```

```
review_date: "January 31, 2024"
```

```
next_meeting: "first Monday of February"
```

## 5.4 Field Precedence and Smart Defaults

md-pdf uses intelligent precedence for configuration values:

1. **Front matter** (highest priority) - Values explicitly set in YAML
2. **Configuration file** - Defaults from your config file
3. **System defaults** (lowest priority) - Built-in fallbacks

## 5.5 Front Matter Example

```
---
title: "Machine Learning Applications in Document Processing"
subtitle: "A Comprehensive Survey and Future Directions"
author: "Dr. Sarah Johnson"
affiliation: "University of Technology"
department: "Computer Science"
date: "2024-01-22"
template: "academic"
toc: true
language: "en"
tags: ["machine learning", "document processing", "natural language processing", "computer vision", "automation"]
abstract: |
  This paper presents a comprehensive survey of machine learning
  techniques applied to automated document processing tasks,
  including natural language processing, computer vision, and
  hybrid approaches. We analyze current methodologies, identify
  key challenges, and propose future research directions.
keywords:
  - machine learning
  - document processing
  - natural language processing
  - computer vision
  - automation
---
```

# 6 Usage Guide

## 6.1 Basic Commands

md-pdf provides a clean, intuitive command-line interface designed for efficiency and ease of use.

```
> md-pdf -h
```

Convert markdown files to PDF using typst

Usage: md-pdf [OPTIONS] [INPUT]

Arguments:

[INPUT] Path to the input Markdown file

Options:

-O, --output <OUTPUT> Path to the output PDF file

## md-pdf

```
-t, --template <TEMPLATE> list the templates and select the one you want [default: none]
-w, --watch                Watch the input file for changes and rebuild automatically
--check-links              Check all links in the markdown file and display warnings for
unreachable links
--list-templates           List all available templates
--create-config            Create default configuration file
--show-config              Show configuration file locations and settings
-h, --help                 Print help
-V, --version              Print version
```

### 6.1.1 Essential Operations

```
# Convert with all defaults (uses config settings)
```

```
md-pdf document.md
```

```
# Specify output filename
```

```
md-pdf document.md -o report.pdf
```

```
# Use specific template (overrides config default)
```

```
md-pdf document.md -t simple -o formatted-report.pdf
```

```
# Watch file for changes (live preview mode)
```

```
md-pdf --watch document.md
```

```
# Watch with custom output and template
```

```
md-pdf --watch document.md -t simple -o live-preview.pdf
```

```
# Watch with custom output and template and check all links
```

```
md-pdf --check-links --watch document.md -t simple -o live-preview.pdf
```

### 6.1.2 Information and Configuration Commands

```
# List all available templates with descriptions
```

```
md-pdf --list-templates
```

```
# Show current configuration and all paths
```

```
md-pdf --show-config
```

```
# Create default configuration file (if not exists)
```

```
md-pdf --create-config
```

```
# Show help information
```

```
md-pdf --help
```

```
# Show version information
```

```
md-pdf --version
```

### 6.1.3 Link Checking and Validation

md-pdf includes built-in link checking functionality to validate external links in your markdown documents before PDF conversion.

```
# Check all external links in document
```

```
md-pdf --check-links document.md
```

#### 6.1.3.1 Sample Output

```
🔍 Checking 5 links in 'research-paper.md'...
```

```
✅ https://github.com/typst/typst
```

```
✅ https://www.rust-lang.org
```

```
❌ https://broken-example.invalid - Error: dns error: failed to lookup address information
```

md-pdf

✓ <https://docs.rs>

✓ <https://wikipedia.org>

📊 Link check summary:

✓ Successful: 4

✗ Failed: 1

⚠ Some links are not reachable. Consider reviewing them.

Using template: simple

✓ Successfully converted 'research-paper.md' to 'research-paper.pdf'